ASSIGNMENT-13.4

NAME: R.SHASHIDEEPIKA

H.NO:2303A51871

B-13

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
doubled.append(v * 2)
print(doubled)

Expected Output

[4, 8, 12, 16, 20]

Prompt:

#Refactor this loop-based Python code into a shorter, Pythonic version using a list comprehension keeping the same output.

Original code:

```
# Original code:
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Refactored code:

```
# Refactored code using a list comprehension:
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code> & C:\Users\shash\AppData\Local\
[4, 8, 12, 16, 20]
[4, 8, 12, 16, 20]
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
```

Justification :

The list comprehension approach improves readability by expressing the transformation in a single, concise line. It clearly communicates the intent—*double each value in the list*—without requiring extra boilerplate code. This makes the script easier to understand and maintain, as future readers can grasp the logic at a glance without tracing through multiple lines of loop-based logic.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""
for w in words:
message += w + " "
print(message.strip())

Expected Output

Refactoring with AI improves quality

Prompt:

#Refactor this Python code that builds a sentence using repeated string concatenation. Make it more efficient and readable with join, while keeping the same output.

Original code:

```
# Original code:
words = ["Refactoring", "with", "AI", "improves","quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Refactored code:

```
# Refactored code:
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/U
Refactoring with AI improves quality
Refactoring with AI improves quality
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The refactored version using join is more efficient because it avoids repeated string concatenation inside a loop. In Python, strings are immutable, so every time you use +=, a new string object is created, which can be costly for larger lists. By using join, all words are combined in a single pass, making the code faster and more memory-friendly.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
else:
print("Default timeout used")
```

Expected Output

Default timeout used

Prompt:

#Refactor this Python code that checks for dictionary keys using if-else. Use a safer and shorter approach like dict.get() to handle missing keys, while keeping the same output.

Original code:

```python
# Original code
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Refactored code:

```python
# Refactored code
config = {"host": "localhost", "port": 8080}
print(config.get("timeout", "Default timeout used"))
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code> & C:\Users\shash\AppData\Local\Programs\Python\Pyt
Default timeout used
Default timeout used
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The refactored version improves readability by reducing the code to a single, clear line. It eliminates the need for verbose if-else statements and makes the intent obvious: *retrieve the value if it exists, otherwise use the default*. This concise style is easier to maintain and immediately understandable to anyone reviewing the configuration loader

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine operations.
• Ask AI to suggest a cleaner, scalable alternative.
• Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
result = x + y
elif action == "subtract":
result = x - y
elif action == "multiply":
result = x * y
elif action == "divide":
result = x / y
else:
result = None
print(result)
```

Expected Output

5.0

Prompt:

#Refactor this Python code that uses multiple if–elif conditions for operations. Use a dictionary mapping approach to make it cleaner and scalable, while keeping the same output.

Original code:

```python
#Original code:
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = "None"
print(result)
```

Refactored code:

```python
#Refactored code:
action = "divide"
x, y = 10, 2
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}
result = operations.get(action, lambda a, b: "None")(x, y)
print(result)
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code> & C:\Users\shash\AppData\Local\Programs\Python\Python3
5.0
5.0
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The dictionary mapping approach is more scalable because new operations can be added by simply extending the dictionary, without modifying multiple if-elif blocks. This makes the code easier to maintain and adapt as the number of supported actions grows, which is especially useful in utility modules that may expand over time.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
if item == "eraser":
found = True
break
print("Item Available" if found else "Item Not Available")

Expected Output

Item Available

Prompt:

#Refactor this Python code that uses a manual loop to search for an item in a list. Use a more concise approach like the in operator, while keeping the same output.

Original code:

```
#original code:
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Refactored code:

```
#refactored code:
inventory = ["pen", "notebook", "eraser", "marker"]
item_to_find = "eraser"
print("Item Available" if item_to_find in inventory else "Item Not Available")
```

Output:

```
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c
Item Available
Item Available
PS C:\Users\shash\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

The refactored version using the in operator is more efficient because it removes the need for a manual loop and extra variables. In the legacy code, the program iterates through each item until it finds a match, which adds unnecessary overhead. By using in, Python performs the membership check directly in a single, optimized step. This approach is not only faster but also scales better when working with larger collections, making it a more practical solution for inventory or data-search tasks.